Donkey Kong: Rush

Final Report for the Distributed Systems Course [A.Y. 2024/2025]

Foschi Giacomo
giacomo.foschi3@studio.unibo.it

Terenzi Mirco
mirco.terenzi@studio.unibo.it

October 14, 2026

This report presents the "Donkey Kong: Rush" project, realized for the "Distributed System" course at the University of Bologna. The project involves the development of a 2D multiplayer platform video game inspired by the classic Donkey Kong. The system is designed around an Entity-Component-System (ECS) architecture written in Java, utilizing JavaFX for rendering. The main focus of the project is the implementation of smooth multiplayer gameplay. The solution combines the host's authority over the deterministic environment with a reactive handling of local inputs, guaranteeing an experience free of blocking lag for the players.

Disclaimer

During the preparation of this work, the authors used GitHub Copilot and DeepL to assist with writing the Javadoc and to check the grammar of this report. Moreover, Copilot has been used as a review tool in some pull requests on GitHub (for example, here). After using this tool/service, the authors reviewed and edited the content as needed and take full responsibility for the content of the final report/artifact.

1 Concept

"Donkey Kong: Rush" is a desktop application featuring a graphical user interface (GUI). Specifically, it is a 2D multiplayer platform video game, which takes inspiration from the original Donkey Kong arcade game.

1.1 Use case description

The software provides a competitive multiplayer platforming experience. Users are located on separate desktop machines connected over the same local area network. Thus, distribution is a fundamental requirement for this project to enable a seamless multiplayer experience, as it allows players on physically separate machines across a network to connect, interact, and share the same virtual space.

Interaction is continuous and in real-time during an active game session. The system captures inputs and exchanges state updates across the network multiple times per second to maintain synchronization.

Players interact with the system via the GUI during setup or keyboard controls during the game. The game controls consist of standard keybindings for movement (left/right, climbing ladders) and jumping.

The system does not require persistent, long-term data storage. All necessary data represents the state of the ongoing match (such as player and barrel coordinates).

The game relies on multiple user roles:

- **Host (Player)**: Actively plays the game, processes their own local input, and acts as the local server. It possesses authority over the game environment (e.g., generating barrels) and broadcasts the world state to all connected clients.
- **Guest (Player)**: Actively plays the game by processing their own input locally while simultaneously receiving continuous updates from the host regarding the rest of the world state.
- **Spectator**: A purely passive role that does not generate game input, but solely receives updates from the players to feed its local rendering system, allowing another user to watch the match in real-time.

2 Requirements Elicitation and Analysis

Glossary

Term	Definition
Host	A player who actively plays the game, processes local input, acts as the authoritative entity for game-world generation (e.g., barrels), and broadcasts the state to clients.
Guest	A player who actively plays the game, processes local input, and receives continuous world state updates from the host.
Spectator	A passive user who does not generate input but receives real-time updates to render and watch the match.
Lobby	The pre-game networking state where users connect and are assigned their respective roles (host, guest, or spectator) before the match begins.
Entity	Any distinct object in the game world.
Barrel	A dynamic entity that deals damage when a player comes into contact with it.
Ladder	A climbable entity in the game level that enables players to move vertically regardless of gravity.
Mario	The playable avatar assigned to the host, capable of moving horizontally, jumping, and climbing ladders.
Luigi	The playable avatar assigned to the guest, sharing identical movement mechanics with Mario while competing in the race to the top.
Pauline	The static goal entity positioned at the top of the level; colliding with her triggers the winning condition and concludes the match.
Donkey Kong	The non-playable antagonist entity positioned at the top of the arena, serving as the source that periodically spawns rolling barrels.

Functional Requirements

Description	Acceptance Criterion
The system must allow users to join a lobby and automatically assign them a role (host, guest, or spectator) based on join order or preference.	A user successfully connects to the server and receives a distinct role assignment (host, guest, or spectator) before the game starts.
The system must allow active players to move horizontally (left/right), jump, and climb ladders.	Pressing the designated keys updates the player's position appropriately on the screen according to game physics (gravity, collision).
The system must synchronize the game state (entities positions, and lives) between all connected clients in real time.	When the host moves or a barrel spawns, the guest and Spectators see the updated positions on their screens without noticeable desynchronization.
The system must detect collisions between players and damaging entities (e.g., barrels), deducting a life upon impact.	When a player's character intersects with a barrel, the player's life count decreases by 1, and the character respawns at the starting position.
The system must declare a winner if a player reaches the goal (Pauline) or declare a loser if a player's lives reach zero.	The game transitions to a "Game Over" screen displaying the correct winner or loser when the goal is touched or 3 lives are lost.

Non-Functional Requirements

These requirements define the behavioral aspects and quality attributes of the system.

Description	Acceptance Criterion
The game must update and render at a consistent frame rate to ensure smooth gameplay.	The local game loop executes consistently at the target of 60 Frames-Per-Second (FPS).
State updates must be transmitted rapidly to prevent visual stuttering or unfair advantages.	State update payloads are serialized, transmitted over the local network, and deserialized by the receiving client.
The system must ensure automatic recovery from disconnections for non-host users.	If a guest or spectator loses their network connection and subsequently reconnects to the server while the match is still active, the system automatically restores their role and resumes sending them real-time game state updates without requiring a full lobby reset.
Code must be modular and documented to make extensions and changes easy.	Modularity is enforced by strictly decoupling data structures from execution logic, avoiding deep and rigid inheritance trees. Maintainability is verified by the presence of comprehensive Javadoc comments on all core API interfaces, ensuring new features can be integrated rapidly.

2.1 Relevant Distributed System Features

- **Evolvability and maintainability:** To ensure long-term maintainability and ease of updates, the system's architecture must decouple game state data from execution logic. The design must allow new game mechanics or objects to be added modularly, avoiding rigid class hierarchies.
- **Fault tolerance, dependability, and availability:** While data integrity for long-term storage is irrelevant since all match state is kept in volatile memory, the system must gracefully handle network faults. If a player's connection drops, the central server must detect the failure and immediately broadcast a game-over message, resetting the lobby to prevent deadlocks and ensure continued availability for future matches.
- **Resource sharing:** The active game world acts as a synchronized shared resource. The host is responsible for managing authoritative game mechanics and synchronizing this shared state with the guest and Spectators via continuous network message broadcasts.
- **Transparency:** The system provides basic *location transparency*; clients seamlessly connect to the lobby without needing to know the physical network topology

of the other players. However, *failure transparency* is intentionally absent; if a connection drops, the failure is explicitly exposed to the remaining users via the game over UI.

- **Scalability:** The game is explicitly bounded to a small, finite number of simultaneous connections (one host, one guest, and passive Spectators) on a local area network. It is not expected to scale horizontally to thousands of users or handle massive data growth over time.
- **Security and trust:** As a local LAN game without persistent or sensitive data storage, there is no need for cryptographic schemes, data encryption, or complex authentication mechanisms.

3 Design

3.1 Architecture

The project follows the Model-View-Controller (MVC) and Entity-Component-System (ECS) architectural patterns. The MVC pattern is used to separate the User Interface (UI) from the game logic, while the ECS pattern is used because it favors composition over inheritance, providing flexibility to add/remove behaviors (components) and logic loops (systems).

Regarding its distributed nature, the system utilizes an event-driven **Client-Server architecture**. Within this framework, the project implements a hybrid synchronization model that combines a *leader-follower* pattern for the environment with distributed authority for player movement. The session creator acts as the authoritative host for the game world, holding exclusive control over global state transitions and environmental entities, such as generating and moving barrels. However, for player characters, the system utilizes client-side authority: each client independently calculates its own character's movement and actions locally, transmitting these authoritative updates to the other peers. This design guarantees a consistent world state while completely eliminating input lag for the players.

3.2 Infrastructure

To support the game's multiplayer requirements, the infrastructure relies on a centralized star topology where all network traffic flows through the main server.

Infrastructural Components

- **Server:** Exactly one server component is active per game session. It manages the game lobby, handles network connections for all participants, assigns roles, and acts as the authoritative source for game state transitions (e.g., starting the game, handling disconnections).

- **Clients:** There are multiple client components per session, one for each playing user and spectator. They establish a connection to the server to forward outgoing inputs/messages and receive game state updates.
- **Message Broker:** A local event bus acts as an internal message broker on each machine. It decouples the network communication layer from the core game logic, routing incoming and outgoing messages between the network handlers and the game systems.

Network Distribution

The game is designed to be deployed and played over a Local Area Network (LAN) and the network distribution of the components depends dynamically on the players' roles:

- **Host machine:** The player who creates the game session acts as the host. His physical machine runs both the central server component and his own local client component.
- **Guest/Spectator machines:** The other participants run only the client component on their respective physical machines. These clients connect remotely over the local network to the host's IP address.

Therefore, the infrastructure is entirely localized within the players' shared local network.

Service Discovery

Components do not rely on static IP configuration. Instead, they dynamically discover active sessions using a custom *service discovery* mechanism based on UDP broadcasts: when a player attempts to join or spectate a game, their client sends a UDP discovery request to the entire subnet. The host's server listens for these packets and replies directly to the sender with the necessary connection details (IP, port, etc.).

Moreover, to avoid scenarios where multiple players attempt to host a lobby simultaneously on the same network, lobbies constantly broadcast their presence. If a server detects another active lobby with a higher priority, it yields its host status, shuts down its server component, and connects as a client.

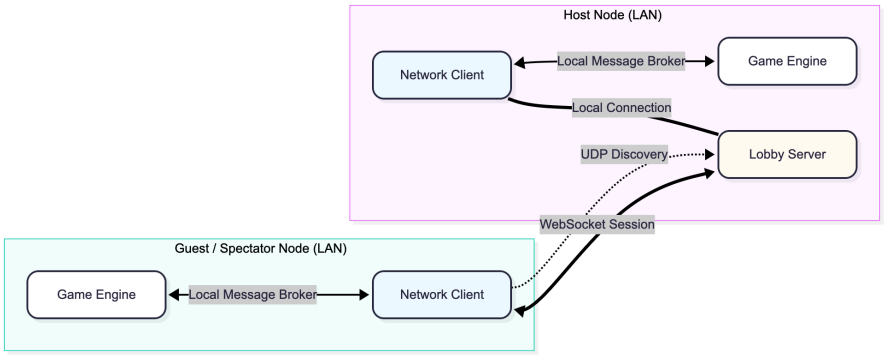


Figure 1: Component Diagram

3.3 Modelling

Domain Entities and Infrastructure Mapping

The domain models the distributed system and its state synchronization through specific entities, which are mapped to the underlying infrastructure based on a partitioned authority model:

- **Game Session (Lobby)**: The core entity representing the match lifecycle. It resides centrally in the server's memory and maps incoming connections to specific network roles (`HOST`, `GUEST`, or `SPECTATOR`). It acts as the single source of truth for the game phase and connection fault tolerance.
- **Player Avatars (Mario & Luigi)**: Synchronized game entities. Rather than a fully centralized model, ownership is distributed: the host computes and broadcasts its avatar's physics, while the guest independently computes and broadcasts its own. Remote clients hold a local replica updated via the ECS receiver.
- **Dynamic Obstacles (Barrels)**: Authoritative game entities. Their generation and physical simulation reside exclusively on the host's infrastructural loop, which guarantees a single source of truth. The server relays this data, and clients map these entities as read-only visual replicas in their local ECS.
- **Static Environment (Map, Platforms, Ladders)**: Local-only entities. They reside entirely in the clients' memory and are loaded by a factory component at startup, requiring no network synchronization.

Distributed System State

The overall state of the application is decoupled into two distinct layers:

- **Session State (server)**: Maintained by the server. It comprehends the registry of active connections, the assigned roles, the current global game phase, and the active reconnection timers.

- **Replicated Game State (client / host)**: The real-time state of the game held within the local ECS of each node. This state comprehends only the strictly necessary data: precise spatial coordinates, visual states (e.g., `IDLE`, `JUMP`), facing directions, remaining lives, and a list of active dynamic obstacles (their `id`, and coordinates).

Domain Events and Exchanged Messages

Communication relies on serialized JSON messages, categorized by interaction type:

1. **Peer Discovery (UDP)**: Messages exchanged to resolve local network topology.
   - `DISCOVER` (Guest → LAN): Client request broadcasted to search for available game sessions.
   - `LOBBY` (Host → Guest): host reply advertising its port, lobby ID, and slot availability.
2. **Session and Fault Tolerance (Server → Client)**: Messages orchestrating the match lifecycle, role delegation, and managing connection drops.
   - `ROLE_ASSIGNMENT`: Instructs a newly connected client of its domain role (`HOST`, `GUEST`, or `SPECTATOR`).
   - `GAME_START`: Commands clients to instantiate the ECS world and begin the simulation loop.
   - `GUEST_DISCONNECTED`: Notifies clients of a connection drop, triggering a 30-second fault-tolerance timer on the server.
   - `GUEST_RECONNECTED`: Signals the successful recovery of the guest's connection within the allowed time frame.
   - `RESTORE_STATE`: Authoritative event forcing a reconnected guest to sync back to specific coordinates and lives to safely resume the session.
   - `GAME_OVER`: Broadcasts the definitive termination of the match, carrying the reason and the winner's name.
3. **Game State and Commands (Client ↔ Server)**: High-frequency state payloads and authoritative triggers mapping in-game events to the network.
   - `HOST_UPDATE`: Broadcasts the Host's avatar state alongside a comprehensive list of all active dynamic obstacles (`BarrelData`).
   - `GUEST_UPDATE`: Broadcasts exclusively the guest's avatar state.
   - `PLAYER_DIED`: Triggered locally when a player's health drops to zero. The server intercepts this and broadcasts a `GAME_OVER` event.
   - `GOAL_REACHED`: Triggered locally when a player successfully collides with the final objective. The server intercepts it and declares the match's end via `GAME_OVER`.

- `ENTITY_DESTROYED`: Event instructing remote clients to remove a specific network entity from their local ECS world.

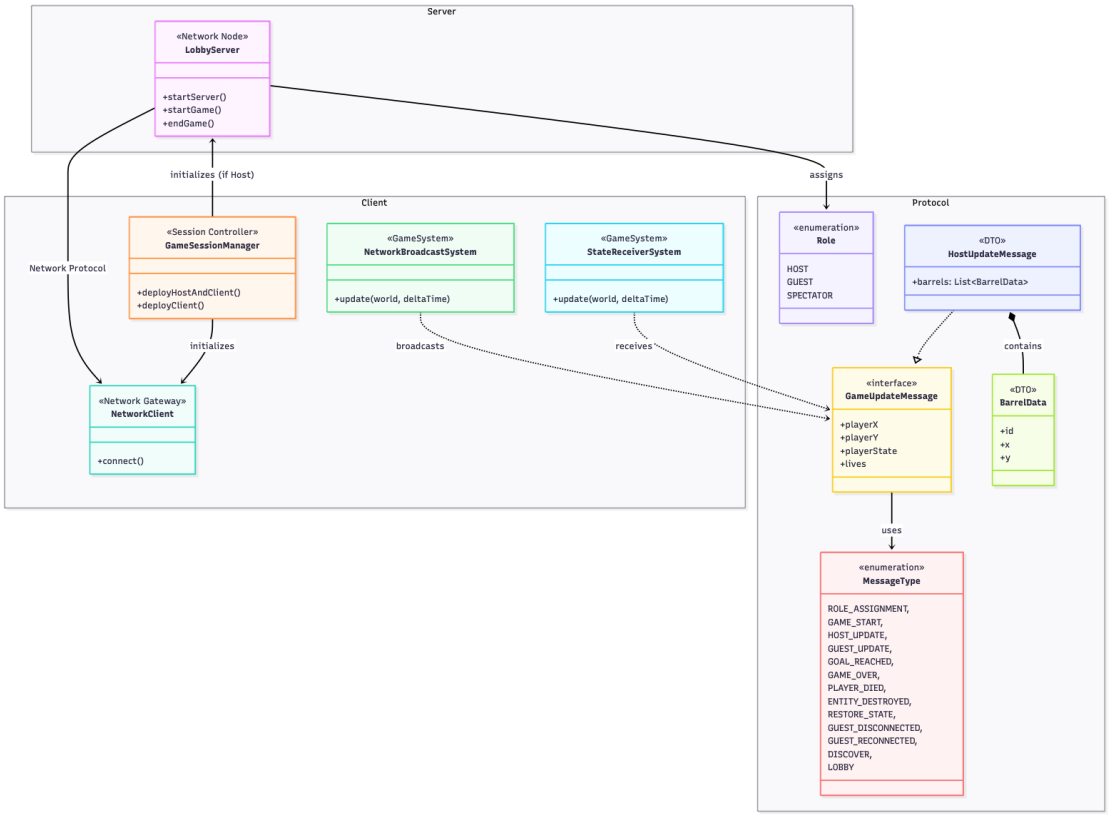


Figure 2: Class Diagram

3.4 Interaction

The system's interaction is event-driven and asynchronous, with the server acting as a central relay for all messages, using both WebSocket for continuous state updates and UDP for initial discovery. The communication patterns naturally adapt to the application's lifecycle, which can be divided into four main phases.

Discovery Phase

The interaction lifecycle begins with network discovery. To find available game sessions, clients periodically broadcast UDP datagrams across the local network. Concurrently, nodes hosting an open lobby listen for these broadcasts and reply immediately upon receiving a valid request, provided there are still open slots in the lobby. Once a lobby is discovered, the system establishes a persistent connection to guarantee stable data

exchange. Clients connect to the central server, which handles the session and assigns specific roles.

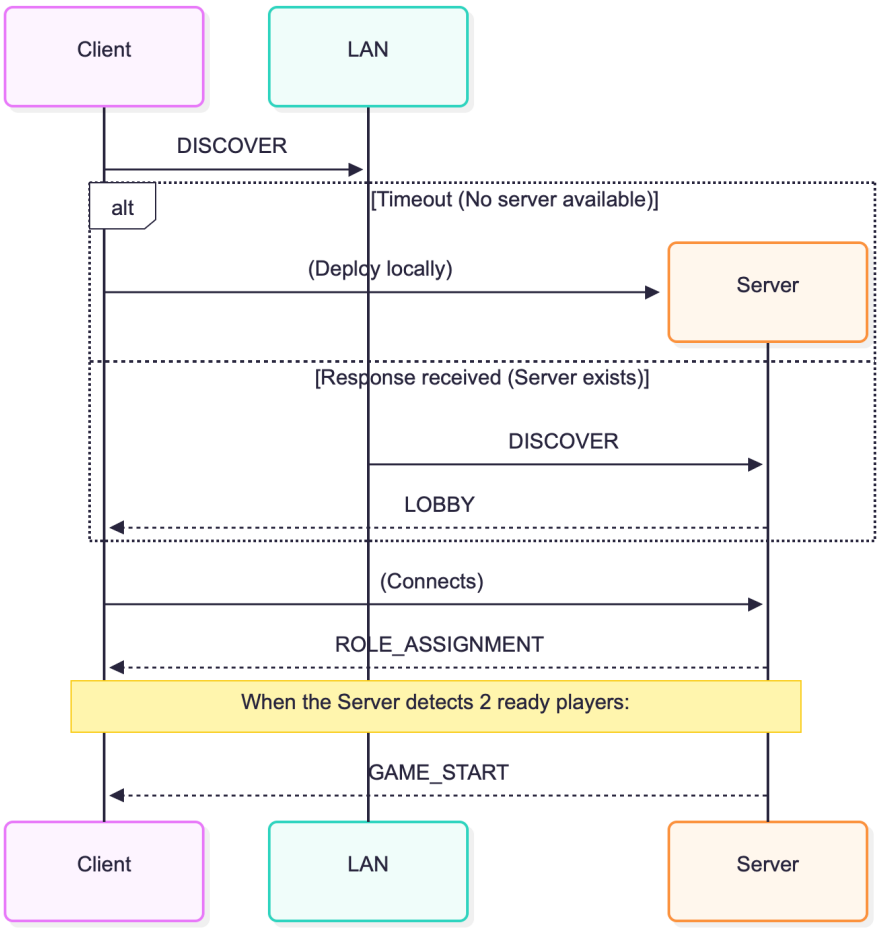


Figure 3: Sequence Diagram - Discovery Phase

Active Gameplay Phase

During the active game phase, communication becomes continuous and is driven by the game loop. At every frame, dedicated broadcasting systems collect the current state of relevant entities and dispatch structured JSON payloads. These payloads include spatial coordinates, current animation states, facing directions, and remaining lives.

To maintain a responsive experience and minimize input lag, the game adopts a split-authority state synchronization pattern. The host dictates the state of the shared environment, such as the spawning and tracking of dynamic obstacles like barrels, alongside its own avatar. Meanwhile, the guest retains authoritative control over its own character's movements. The central server continuously relays these updates to the oppos-

ing player and broadcasts them to all connected spectators. Alongside this continuous stream, asynchronous events (e.g., a player reaching the goal or losing a life) trigger the dispatch of specific control messages to notify all connected nodes.

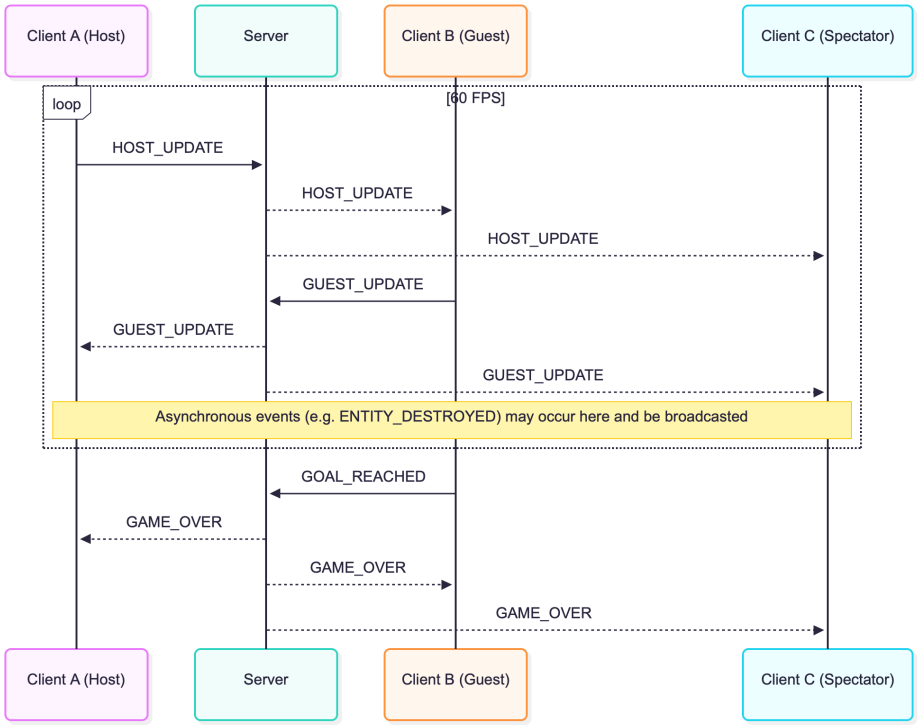


Figure 4: Sequence Diagram - Active Gameplay Phase

Recovery Phase

The architecture is designed to handle unexpected network interruptions. Disconnection and reconnection trigger notification messages across the system. If a disconnected guest reconnects within the allowed timeframe, the host dispatches a synchronization payload to restore the guest's last known coordinates and lives, resuming the game seamlessly. Finally, when a winning or losing condition is met, the central server broadcasts a game-over control message. This event triggers the UI observers, transitioning players to the final summary screens and tearing down the active game session.

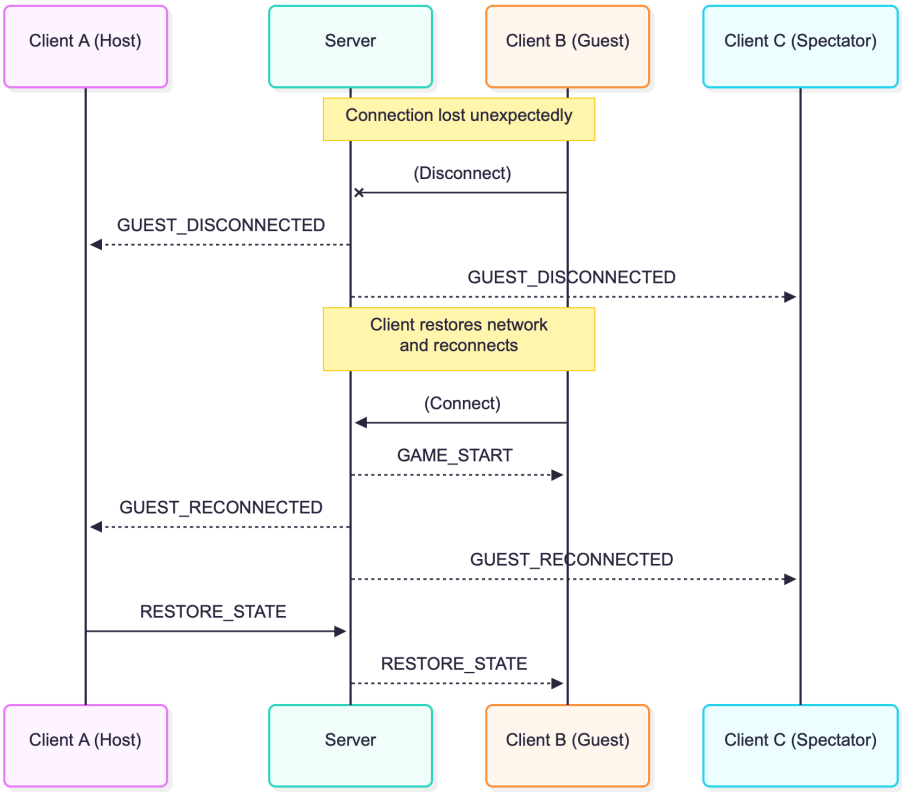


Figure 5: Sequence Diagram - Recovery Phase

3.5 Behaviour

The system behavior is modeled as an event-driven architecture where the server acts as the central Finite State Machine (FSM), while clients react to the server's state transitions.

Server

The server behavior is modeled through four states, each representing a distinct phase of the game session:

1. `WAITING_PLAYERS`: The initial state of the server immediately after the lobby is created. The server has a host (or is waiting for one to connect), but the guest is missing. In this state, the UDP Discovery mechanism broadcasts that the guest slot is available. Spectators can join during this phase without triggering transitions.

2. `GAME_RUNNING`: The state in which the match is active. The host and the guest continuously exchange their respective state updates.

3. `WAITING_RECONNECT`: A fault-tolerance state. It occurs when, during an active match, the guest drops the connection. The game "freezes" for the guest, while the server waits for a maximum timeout (30 seconds). If the guest reconnects within this window, the server restores its last known state and resumes the match. If the timeout expires, the server declares a connection loss (without winner) and transitions to `GAME_OVER`. Similarly, if the host disconnects, the match is terminated with a connection loss for all participants.

4. `GAME_OVER`: The terminal state where the match ends due to victory, defeat, or definitive disconnection. The server can either reset the lobby to `WAITING_PLAYERS` or shut down entirely.

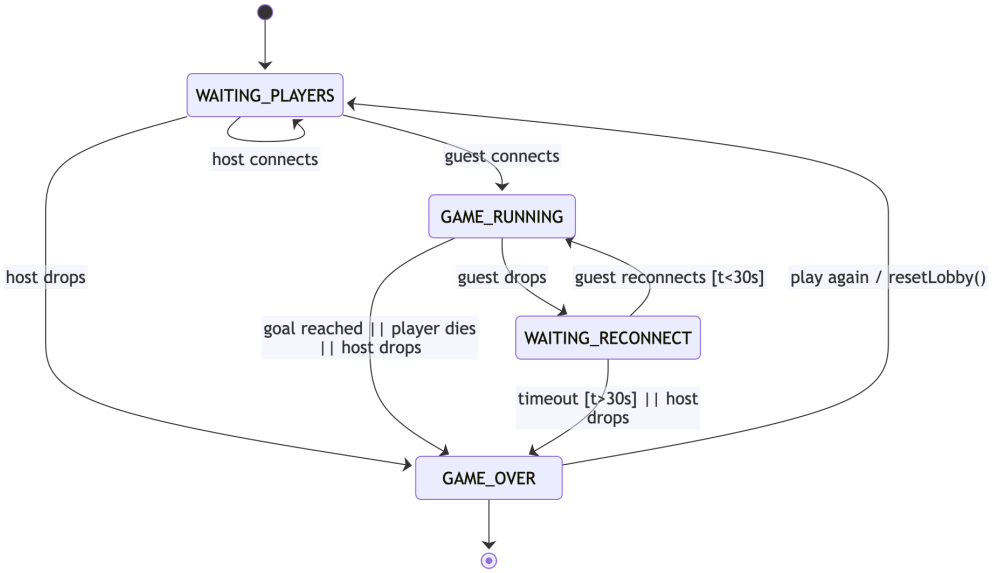


Figure 6: State Diagram

Client

The client is modeled as a reactive state machine driven by messages received from the server and events generated by the local game loop. Its behavior differs according to the role assigned during the lobby phase, but all clients share the same lifecycle:

1. `DISCONNECTED`: The client is not connected to a session. A player searches for an available lobby through UDP discovery, while a spectator searches for an active game to observe.

2. `CONNECTING`: After discovering a lobby, the client establishes a connection and waits for a `ROLE_ASSIGNMENT` message. The assigned role determines which messages the client may send during the match.

3. WAITING_FOR_GAME: The client has joined the lobby and waits for GAME_START. It initializes the local interface and prepares the ECS world, but does not yet process gameplay input.

4. PLAYING: Upon receiving GAME_START, the client starts the game loop. An active player converts keyboard input into local events and updates its own avatar immediately. The host also simulates the shared environment, including barrel generation and movement. At regular intervals, it sends HOST_UPDATE messages containing its avatar and barrel state, while the guest sends GUEST_UPDATE messages containing only its own avatar state. Spectators do not send gameplay updates.

   During PLAYING, local movement is intentionally handled without waiting for a network round trip, eliminating input latency for the controlled avatar. Remote entities are updated from the most recent network snapshot, and incoming authoritative coordinates overwrite stale local values to preserve eventual consistency. Local collision and goal events are published as PLAYER_DIED or GOAL_REACHED messages; the server validates the resulting session transition and broadcasts the final outcome to every connected client.

5. WAITING_RECONNECT: If the guest loses its connection, its local gameplay is suspended while the client attempts to reconnect to the session. A successful reconnection causes the client to receive a RESTORE_STATE message with the avatar's coordinates and remaining lives, after which the world and game loop resume. To detect this connection drop locally, the guest and spectator clients monitor the incoming HOST_UPDATE messages. If no updates are received for 1.5 seconds, the clients assume a network drop and freeze their local rendering to prevent divergent gameplay. If the connection remains broken and no messages arrive for 40 seconds, the clients locally time out, clean up the session, and force a transition to GAME_OVER.

6. GAME_OVER: When the client receives GAME_OVER, it stops processing gameplay updates, displays the result identified by the server, and releases the active game resources. The user can then return to the main menu or exit the application.

3.6 Fault-Tolerance

- **State Replication and Authority:** The game uses selective state replication to keep the participating clients visually synchronized without distributing full control of the simulation to every node. The host is authoritative for the shared environment, including dynamic obstacles and its own player state. The guest reports the state of its controlled player to the host, while spectators receive replicated updates in read-only form. This arrangement limits conflicting updates and provides a clear authority model for resolving state changes.

- **Connection Loss and Recovery:** A temporary loss of connectivity by the guest does not immediately terminate the match. The active session enters a recovery

window during which the guest's last known state is retained and the remaining participants are informed of the interruption. On guest side, he actively monitor his connection status by tracking incoming host updates. If no updates arrive for 1.5 seconds, they freeze their local simulation to prevent the game from continuing in an isolated, desynchronized state. If the connection is lost for more than 40 seconds, the clients locally time out and terminate the match. If the guest reconnects in time, the session restores the last synchronized player state and resumes normal communication. This approach tolerates short-lived network interruptions while avoiding uncontrolled divergence between the two players.

- **Component Failure:** The host is a critical component because it maintains the authoritative shared simulation. Its disconnection cannot be recovered without risking inconsistent game state, so the current match is terminated and the session is reset. A spectator failure has no effect on the match: spectators are passive consumers of replicated state and can leave without affecting either player or the simulation.

3.7 Availability

- **Resource Reuse:** Frequently accessed visual resources are retained locally after they have been loaded and prepared for rendering. Reusing these resources avoids unnecessary I/O and processing during the game loop, supporting smooth interaction and consistent visual performance.

- **Network Partitions:** The CAP theorem states that any distributed data store can simultaneously provide at most two of three guarantees: consistency, availability, and partition tolerance. Since the system is designed to operate over a local area network, it must tolerate network partitions and communication failures. In this case, the system prioritizes **consistency over availability** for the affected match: gameplay does not continue independently while communication is interrupted, because doing so could create divergent positions, collision results, or victory conditions and give a player an unfair outcome. Instead, the session enters recovery and resumes only from a synchronized state, or is terminated if recovery is unsuccessful. A spectator leaving the session does not affect the match because spectators do not participate in the simulation. By contrast, loss of the authoritative participant cannot be safely recovered without risking inconsistent state, so the affected match is ended.

4 Implementation

This chapter details the specific technology-dependent choices made to realize the architectural design, focusing on network protocols, data serialization, and the frameworks exploited.

- **Network Protocols:** The system employs a dual-protocol approach to handle different networking phases efficiently:
  - **UDP (User Datagram Protocol):** Utilized exclusively for the initial **service discovery** phase and conflict resolution. Clients broadcast `DISCOVER` datagrams on port 8081 across the LAN, and available hosts reply directly with the WebSocket connection details.
  - **WebSockets (WS) over TCP:** Utilized for all continuous in-game communication. WebSockets were chosen because they provide a persistent, full-duplex communication channel with guaranteed, ordered delivery out-of-the-box. This drastically simplifies the implementation for a local area network (LAN) environment, ensuring the server can reliably push state updates to all clients.
- **In-transit Data Representation:** All data exchanged over the network is serialized and represented in **JSON**. At the implementation level, the system leverages Vert.x's built-in `JsonObject` to automatically map incoming and outgoing JSON payloads directly to immutable Java `record` classes (e.g., `HostUpdateMessage`, `GuestUpdateMessage`). This choice eliminated boilerplate parsing code while maintaining high human-readability, which greatly sped up the debugging process.
- **Data Storage and Persistence:** The system operates entirely without a traditional Database Management System (DBMS) or persistent file storage. Due to the session-based, arcade nature of the game, all state data—such as entity coordinates, physical velocities, and life counts—is maintained exclusively in volatile memory (RAM) within the local ECS `World` instance. Once a match concludes or the central server shuts down, this ephemeral game state is completely flushed.
- **Authentication and Authorization:** The system prioritizes immediate accessibility and does not employ strict authentication protocols (such as OAuth or JWT). Players are implicitly trusted upon successfully establishing a WebSocket connection. However, authorization is securely managed via a lightweight Role-Based Access Control (RBAC) enforced programmatically by the server. Roles are deterministically assigned based on the requested URI and chronological connection order: the first user connecting to the `/play` endpoint is granted the authoritative host role, the second becomes the guest, and any subsequent users (or those connecting directly to the `/spectate` endpoint) are restricted to read-only spectator access.

4.1 Technological Details

The project relies on a specific technology stack to achieve its concurrency and rendering goals:

- **Java (JDK 17+):** The entire application is written in Java, taking advantage of modern language features. In particular, Java `record` classes are exploited to de-

fine immutable ECS Components (e.g., `PositionComponent`, `VelocityComponent`) and network messages, ensuring thread safety.

- **Eclipse Vert.x:** This is the core framework used for networking and concurrency. Vert.x is built on a non-blocking, event-driven architecture (using the Reactor pattern), which allows it to handle multiple concurrent connections with minimal thread overhead.
  - A dedicated component acts as the central HTTP/WebSocket server.
  - Specialized network agents utilize Vert.x's `DatagramSocket` for UDP broadcasting.
  - The internal **Vert.x EventBus** is used in the application to decouple the network layer from the game logic layer. Incoming network messages are published to specific EventBus addresses (e.g., `inbound.guest_update`), where the dedicated ECS receiver system consumes them asynchronously, preventing network operations from blocking the main game loop.
- **JavaFX:** Used exclusively for the client-side graphical user interface (GUI) and rendering. The game utilizes a raw `Canvas` and a `GraphicsContext` to manually draw and clear sprite sheets frame-by-frame. The core game loop is driven by a JavaFX `AnimationTimer`, which triggers the ECS engine's main update cycle to process physics and render the graphics concurrently at a targeted 60 FPS.
- **Custom ECS Engine:** The project implements a custom Entity-Component-System engine from scratch. This allows for total control over the modularity of the code, strictly separating pure data representations from the core execution logic, making the implementation of network state synchronization straightforward and predictable.

5 Validation

To guarantee the overall reliability, state consistency, and fault tolerance of the distributed architecture, the validation phase was conducted through a combination of automated unit and integration tests, alongside manual acceptance testing.

5.1 Automatic Testing

Automated testing was implemented using `JUnit 5` and managed via the Gradle build tool, and can be run via `./gradlew test`. These tests are also used to validate the codebase within continuous integration pipelines ensuring that every pull request is validated before being merged.

- **Unit Testing:** Individual components were tested in isolation to verify their correct behavior. In particular:

  - `WorldTest` verifies the core ECS mechanics (adding/removing entities and components).
  - `FactoryTest` ensures entities are assembled with the correct components.
  - All the system-level tests (`PhysicsSystemTest`, `GravitySystemTest`, etc.) are used to verify each system's behavior in isolation, including edge cases like boundary clamping and event dispatching, by using a real, in-memory `WorldImpl` instance and manually invoking the `update` method.

  These tests are useful to verify the functional requirements related to horizontal movement, jumping, collision detection, etc., as well as the modularity non-functional requirement.

- **Integration Testing:** Components were tested together to verify their correct interaction, and how they integrate with external dependencies (e.g., Vert.x).
  - `EndGameScenariosTest` verifies direct win and death callbacks and guest disconnection handling.
  - `LobbyVerticleTest`, `ClientVerticleTest`, and `DiscoveryResponderTest` verify isolated network and discovery behavior, including WebSocket role assignment and broadcasting, client message forwarding through the event bus, and UDP discovery responses.
  - `StateReceiverSystemTest` and `NetworkBroadcastSystemTest` verify the correct integration between the network and the game loop, ensuring that messages arriving on the event bus update the local ECS entities and that outgoing state payloads are correctly formatted.

5.2 Acceptance Test

While automated tests comprehensively covered the programmatic logic and network routing, the real-time distributed nature of the game required manual End-to-End (E2E) testing in a production-like LAN environment with multiple physical machines. Evaluating subjective quality metrics—such as the smoothness of the JavaFX rendering and the tactile responsiveness of controls—alongside simulating physical network adapter disconnections to validate the fault-tolerance window, necessitated human observation and manual intervention.

- **User Interface Flow:** The seamless transition from the main menu to gameplay and, finally, to the game-over screen.
- **Network Synchronization:** The visual coherence of the game state between the host, guest, and spectator screens. This involved verifying that barrel spawns and player movements did not suffer from visual desynchronization.
- **Gameplay Feel:** The responsiveness of the keyboard inputs and the game mechanics (jump height, movement speed).

- **Fault Recovery:** Physically disconnecting the Wi-Fi on the guest machine to verify the avatar freezing behavior, and the correct state restoration upon reconnection.

6 Deployment

Prerequisites
Before deploying the software, the target machine must meet the following software requirements:

- **Java Development Kit (JDK):** Version 17 or higher must be installed.
- **Network:** An active Local Area Network (LAN) connection if multiplayer capabilities are to be utilized and no firewall rules that block UDP broadcasts or WebSocket connections.

To deploy the application, users can download the pre-built JAR file from the release section and execute it directly using `java -jar donkeykong-fat.jar` in the same directory where the JAR is located.

Alternatively, the source code can be cloned from the repository:

```
git clone https://github.com/mircoterenzi/donkey-kong
cd donkey-kong
```

and the application can be run directly using the Gradle wrapper, using `./gradlew run` on Linux/macOS or `gradlew.bat run` on Windows. This approach is particularly useful for developers who wish to modify the code or contribute to the project.

7 User Guide

1. **Main menu:** Upon launch, the user is presented with a graphical interface offering two options:
   - **Play:** Initiates the matchmaking process to either host a new game or join an existing one. If no active game is found, the user will automatically become the host.
   - **Spectate:** Searches for an active game session to observe without participating.

   Note that, if the lobby is already full (i.e., two players are connected), the system will automatically redirect the user to the spectate mode, ensuring that he can still enjoy the game as an observer.

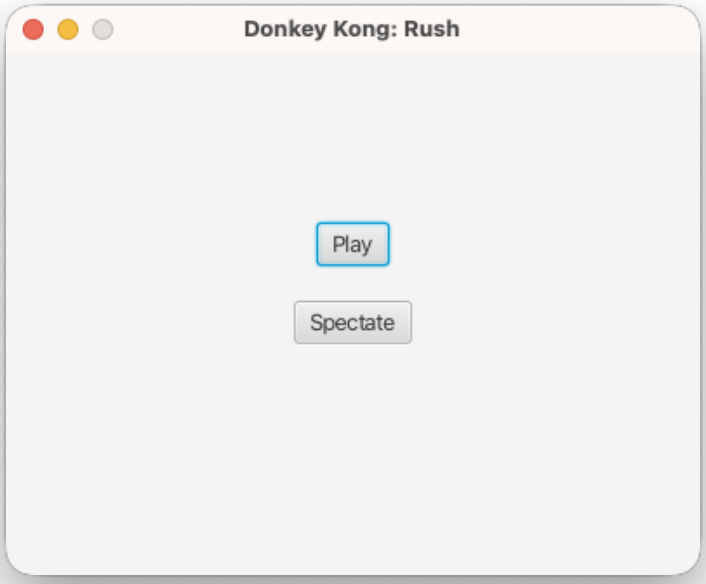


Figure 7: Main menu

2. **In-game controls:** Once the game has started, active players can interact with the environment using the following keyboard bindings:
   - **Movement:** ← / `A` to move left, → / `D` to move right.
   - **Climbing:** ↑ / `W` to climb up ladders, ↓ / `S` to climb down.
   - **Jumping:** `spacebar` to jump over obstacles and gaps.

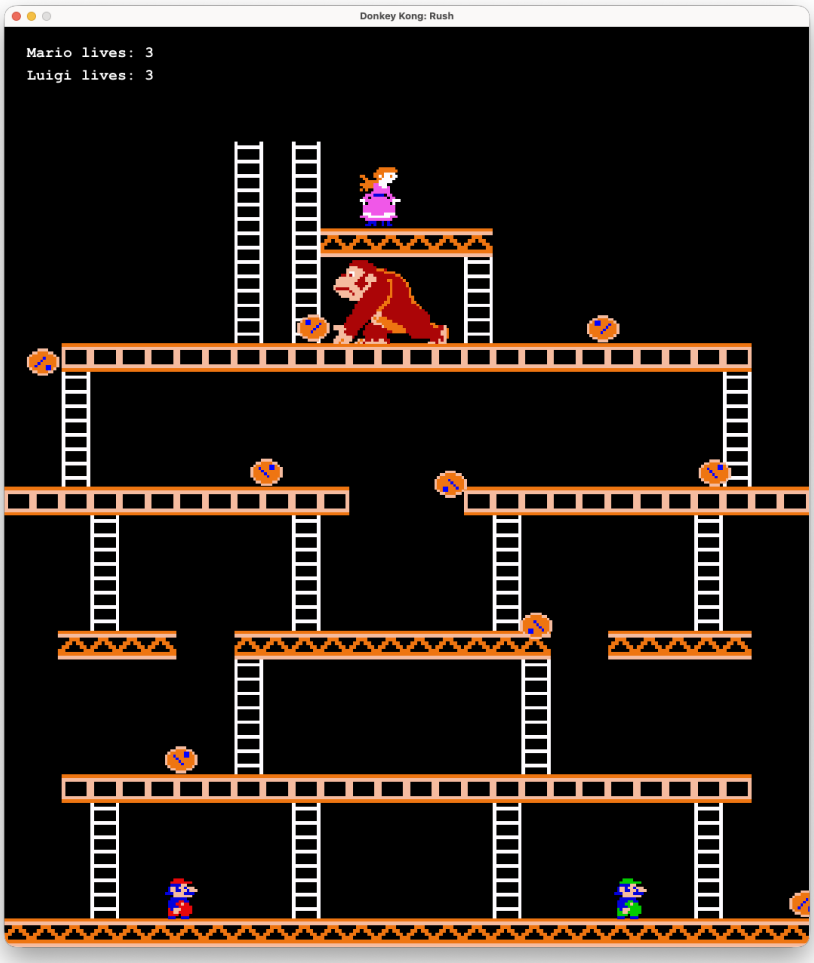


Figure 8: Active Gameplay Arena

3. **Game objective:** Navigate your avatar from the bottom of the screen to the top to rescue Pauline. Avoid the rolling barrels spawned by Donkey Kong. If you collide with a barrel, you will lose a life and respawn at the bottom. The game ends when a player reaches Pauline or loses all 3 lives.

4. **Game-over menu**: Upon reaching a terminal condition (victory or defeat), the game transitions to a summary screen displaying the outcome and offering options to either return to the main menu or exit the application.

8 Self-evaluation

8.1 Foschi Giacomo

I am highly satisfied with the outcome of this project. Building a real-time multiplayer game from scratch required meticulous planning, but it ultimately provided a deep understanding of distributed synchronization and network routing.

Regarding the project's strengths, I consider the fault-tolerance mechanisms and the seamless LAN matchmaking to be the highlights of our work, resulting in a highly accessible and resilient application, alongside the overall fluidity and responsiveness of the gameplay. The primary weakness of the product is tied to its network topology, where the authoritative server is hosted directly on one of the client machines. While perfectly suited for a small LAN environment, scaling the system to a wide area network (WAN) or supporting concurrent multi-lobby sessions would require extracting the backend logic into a dedicated, standalone microservice.

Within the group, I had an excellent and highly productive experience collaborating with Terenzi. While he focused on the core ECS architecture and the discovery mechanism, my specific contributions centered on the structural network architecture, fault tolerance, and validation. I took charge of engineering the core network communication layer using Eclipse Vert.x. I was responsible for defining the message exchange protocols, managing the EventBus routing, and successfully decoupling the asynchronous network payloads from the synchronous game engine. Furthermore, I led the validation phase through rigorous testing and implemented the client-side fault-tolerance logic to ensure smooth recovery from network drops.

8.2 Terenzi Mirco

I enjoyed working on this project because it allowed me to explore both distributed-systems concepts and real-time multiplayer game development. I am particularly satisfied with the final result and with the way the different architectural components were integrated.

Initially, I focused on the design and implementation of the Entity-Component-System architecture and the core game logic. I contributed to defining how entities, components, and systems interact, keeping the game logic modular and extensible. In particular, I worked on the movement, physics, gravity, collision-detection and collision-resolution systems, as well as on animation and game-state management. Regarding the distributed aspects of the project, one of my main contributions was the design and implementation of an automatic LAN lobby-discovery mechanism. Instead of requiring players to manually exchange IP addresses and configure connection details, clients can search the local network for available game sessions. When a suitable lobby is found, the client connects automatically; otherwise, a new lobby is created and the player becomes the host. The same mechanism also supports spectator discovery and includes a conflict-resolution procedure for handling multiple lobbies started simultaneously. I also contributed to the network communication layer, which manages the exchange of messages between the host, guest, and spectators. This layer is responsible for forwarding game-state updates,

broadcasting information to spectators, synchronizing player and barrel state, handling game-start and game-over events, and managing temporary guest disconnections and reconnections.

Overall, I believe that the main strength of the project is its modularity. The ECS architecture separates data from behavior, while the networking and user-interface responsibilities are divided into dedicated components. This makes it possible to add new features or modify existing ones without requiring extensive changes to unrelated parts of the system. Moreover, regarding network synchronization, since the project follows a hybrid authority model: each client simulates the movement and physics of its own player locally and player collisions and damage are evaluated locally instead of relying on a strictly server-authoritative model. This improves responsiveness and makes gameplay smoother by reducing visible latency effects, such as rubber-banding during movement and jumps. The main drawback of this approach is the trade-off between responsiveness and security. Since collision, damage, and death decisions are made by the clients themselves, a malicious client could potentially tamper with its local game state or send fraudulent events. A stricter server-authoritative architecture would provide stronger protection against cheating, but would also introduce additional latency and complexity.

9 Future works

While the current implementation successfully fulfills the core requirements of a distributed, real-time multiplayer platformer, several avenues for improvement and architectural expansion remain.

- Currently, the guest and spectator clients act as "dumb terminals" for external entities, strictly overwriting their local entity coordinates with the incoming network snapshots from the host. On a high-latency network, this could lead to visual stuttering or "rubber-banding." A primary future improvement would be implementing **entity interpolation** (smoothing the visual transition between the last known network state and the current one) and **client-side prediction** (allowing the guest to predict the physics locally, correcting them only if needed).

- The current zero-config matchmaking relies on UDP broadcasting, which strictly bounds the system to a LAN environment. To scale this into a production-grade application playable over the public Internet, the server could be decoupled and deployed as a standalone microservice on a cloud provider (e.g., AWS or Google Cloud) using Docker and Kubernetes. Introducing a centralized matchmaking service would allow clients from different networks to be dynamically paired into isolated game instances orchestrated by the cloud provider.

- From a gameplay perspective, the game currently features a single, hardcoded map generated upon startup via the `MapFactory`. Future iterations could introduce dynamic map loading by parsing external configuration files (e.g., JSON or Tiled

map formats). This would require extending the network protocol to broadcast a `LEVEL_LOAD` message, ensuring that all distributed clients successfully load and synchronize the same level assets before the `GAME_START` event is triggered.

- A possible future extension would be introducing a dedicated persistence layer backed by a database management system (DBMS) to manage **user profiles**, account authentication, and player statistics. Building on this infrastructure, a centralized **leaderboard** service could be implemented to track and rank competitive performance metrics, such as the time elapsed to rescue Pauline or the remaining lives upon victory.